

Module : Kotlin (M202)

TP N° 23

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM101 - DEVOAM102 Niveau : 2ème année

Exercice 1 :

1. Classes et héritage :

- Créer une hiérarchie de classes pour un système de paiement. La classe de base **Paiement** doit avoir une méthode **payer()**.
- Créer deux sous-classes **CarteDeCrédit** et **PayPal** qui redéfinissent la méthode **payer()** avec des comportements spécifiques.

2. NullSafety :

- Créer une fonction qui prend une liste nullable d'entiers et renvoie la somme de tous les éléments non nuls.
- Utiliser des opérateurs de null safety pour éviter les erreurs de null pointer.

3. Interfaces :

- Créer une interface **Réparable** avec une méthode **réparer()**.
- Implémenter cette interface dans deux classes, **Voiture** et **Ordinateur**, avec des comportements de réparation spécifiques pour chaque classe.

4. Objets Kotlin :

- Créer un objet singleton **GestionnaireDeSession** qui contient une méthode **démarrerSession()** et une propriété pour suivre l'état de la session (ouverte ou fermée).
- Simuler l'ouverture et la fermeture d'une session dans un programme.

5. Sealed class :

- Créer une classe scellée **ÉvénementRéseau** avec trois sous-classes : **Succès**, **Erreur**, et **Chargement**.
- Créer une fonction qui prend un objet de type **ÉvénementRéseau** et renvoie un message différent selon le type d'événement.

6. Data class :

- Créer une data class **Produit** avec des propriétés nom, prix, et quantité. Implémente une méthode pour appliquer un rabais au produit et renvoie une nouvelle instance avec le prix actualisé.

7. Nested class :

- Créer une classe **Livre** avec une classe imbriquée **Chapitre**. Chapitre doit avoir des propriétés comme titre et numéro. Implémente une méthode dans Livre pour ajouter un chapitre et afficher les détails de tous les chapitres.

8. Enum :

- Créer un enum **NiveauDePriorité** avec les valeurs **Basse**, **Moyenne**, et **Haute**. Ajoute une propriété qui retourne une couleur associée à chaque niveau.
- Utiliser cet enum dans une classe **Tâche** qui contient une priorité.

9. Extensions :

- Ajouter une fonction d'extension à la classe **Int** pour vérifier si un nombre est un multiple d'un autre entier.
- Utiliser cette extension dans une fonction pour **filtrer** une liste d'entiers et renvoyer uniquement les multiples d'un certain nombre.

10. Collections et ranges :

- Créer une fonction qui prend une liste d'entiers et renvoie la liste triée par ordre décroissant, en excluant les nombres impairs.
- Utiliser un range pour ajouter une fonctionnalité de pagination.

11. Expressions et opérateurs :

- Créer une classe Rectangle avec des propriétés largeur et hauteur, et surcharge l'opérateur + pour ajouter deux rectangles en fonction de leur aire.

12. Appels asynchrones :

- Utiliser async et await pour appeler deux API simulées (avec **delay()** pour simuler des délais réseau), puis renvoie la somme des résultats.

13. Lazy loading :

- Créer une classe **Utilisateur** avec une propriété **profilComple** qui est chargée paresseusement. Simule le chargement des données du profil avec un délai, puis affiche les informations lorsqu'elles sont chargées.

14. Lambda expression :

- Créer une fonction qui prend une liste de **Personne** (avec nom et âge), et utilise une lambda pour trier les personnes par âge. Retourne les personnes triées.

15. Gestion des exceptions :

- Créer une fonction **diviser** qui prend deux entiers et renvoie le résultat de la division. Gère les exceptions de division par zéro et renvoie un message d'erreur approprié.

16. High-order Functions :

- Créer une fonction **appliquerRéduction** qui prend une liste de **Produit** et une fonction **lambda** en paramètre. Applique la réduction définie par la lambda à chaque produit.

17. Fonctions inline :

- Créer une fonction **inline** chronométrer qui prend une lambda, exécute la lambda et affiche le temps d'exécution en millisecondes.

18. Réflexion :

- Utiliser la réflexion pour parcourir toutes les méthodes et propriétés d'une classe **Personne** à l'exécution et les afficher dynamiquement.

19. Type safe builders :

- Créer un type-safe builder pour construire une structure XML simple avec des éléments **div**, **span**, etc., qui sont organisés de manière hiérarchique.

20. Type Aliases :

- Créer un alias de type pour une fonction **Calculatrice** qui prend deux entiers et renvoie un entier. Utilise cet alias dans une classe pour appliquer plusieurs opérations arithmétiques.

21. Coroutines Scope :

- Créer une coroutine qui utilise un **CoroutineScope** personnalisé pour exécuter plusieurs tâches en parallèle, avec une gestion des erreurs intégrée.

22. Fonction launch :

- Utilise **launch** pour démarrer plusieurs coroutines qui simulent des tâches réseau. Annule une de ces coroutines après un certain délai.

23. Fonction **await** :

- Créer une fonction qui utilise **await** pour récupérer des résultats d'opérations asynchrones (simulées avec `delay()`) et les combine.

24. **Jobs** :

- Créer une coroutine avec un **Job** et utilise **cancel()** pour l'arrêter après un certain temps. Vérifie le statut du Job (actif ou annulé).

25. **Dokka** :

- Documente une classe `Produit` et une fonction `calculerRabais` en utilisant des commentaires `KDoc`, puis configure `Dokka` pour générer la documentation du projet.

26. **KDoc** :

- Documente une fonction `calculer` qui prend deux entiers et renvoie leur produit. Utilise `KDoc` pour décrire les paramètres et le comportement de la fonction.

27. **Kapt** :

- Utilise `Kapt` pour créer une simple annotation de validation sur une propriété d'une classe `Utilisateur`, et fais en sorte que l'annotation vérifie que le nom de l'utilisateur n'est pas vide.

28. **Gradle** :

- Ajoute une tâche personnalisée à un projet `Gradle` qui affiche un message "CMC - OFPPT" lors de l'exécution.

29. Notions de l'injection des dépendances :

- Implémente un exemple d'injection de dépendances sans framework.
- Créer une classe Service et injecte-la manuellement dans une classe Contrôleur.

30. Utilisation de Hilt :

- Utiliser Hilt pour injecter un Repository dans une Activity.
- Le Repository retourne juste un message fixe.
- Afficher ce message dans un TextView.